Martin Gonzalez Rico

Colecciones y genéricos

No.	Respuesta	Explicación:
1	b.A collection that cannot contain duplicate elements.	Un Set es una colección que no permite guardar elementos duplicados.
2	a.ArrayList permite elementos duplicados.	ArrayList es una lista que mantiene el orden en el que agregamos los elementos y permite repetir valores.
3	c.Es posible que tengan valores duplicados.	Tanto los arreglos como los ArrayList permiten guardar elementos repetidos.
4	c.Dictionary	Dictionary es una clase abstracta antigua de Java, anterior al Java Collections Framework. En cambio, ArrayList, List y Map sí forman parte de este framework.
5	d.equals()	equals() y hashCode() deben trabajar de manera coherente. Si dos objetos son iguales según equals(), deben devolver el mismo hashCode().Sobrescribir solamente hashCode() sin mantener esa relación puede provocar comportamientos incorrectos al buscar o guardar objetos.
6	a.Apple Banana Cherry Size: 3 Contains 'Banana': true Element at index 1: Banana	Primero se agregan tres elementos a la lista, por lo que size() devuelve 3. El método printList() recorre e imprime los elementos en el orden en que fueron agregados. Después, contains("Banana") devuelve true porque ese valor existe en la lista. Por ultimo, get(1) devuelve "Banana" porque los índices empiezan en 0, así que el índice 1 corresponde al segundo elemento.
7	b.[3, 5, 6, 8]	Collections.sort() ordena la lista usando el orden natural de sus elementos. Como estamos trabajando con números enteros (Integer), los ordena de menor a mayor. La lista originalmente contiene [8, 5, 3, 6], pero después de ordenar queda [3, 5, 6, 8].

8	c.public class Box<T> {
	private T value;
	public Box(T value) {
	this.value = value;
	}
	public T getValue() {
	return value;
	}
	}
	d.public class Box<T extends Number> {
	private T value;
	public Box(T value) {
	this.value = value;
		La opción c utiliza Box<T>, donde T representa un tipo que se define cuando creamos el objeto. La opción d también es válida, pero utiliza T extends Number, lo que limita los tipos permitidos a Number y sus subclases, como Integer, Double y Float. Las opciones a y b no son clases genéricas porque utilizan un tipo fijo (Object o int) en lugar de un parámetro de tipo.

	}
	public T getValue() {
	return value;
	}
	}	

9	b.LinkedHashSet	LinkedHashSet no permite elementos duplicados y mantiene el orden en el que se insertaron. HashSet no garantiza el orden de inserción y TreeSet ordena los elementos según su orden natural o un comparador.
10	d.Imprimirá los artículos ordenados por nombre.	La clase Item implementa Comparable<Item> y define el método compareTo() comparando this.name con other.name. Por eso, cuando se ejecuta Collections.sort(items), los objetos se ordenan alfabéticamente por su nombre, no por su precio.
11	a.Apple: 3 Banana: 2 Cherry: 1	El método sobrescrito de la clase Comparable: compareTo() compara los nombres de los objetos y determina el orden alfabético. Collections.sort(items) utiliza ese método para ordenar la lista. Después, el for recorre la lista ya ordenada e imprime el nombre y el precio de cada objeto.
12	d.7 5 3 2	La clase MySort implementa Comparator<Integer> y su método compare() devuelve y.compareTo(x), invirtiendo el orden natural. Por eso Arrays.sort(primes, ms) organiza los números de mayor a menor. El arreglo empieza como {2, 7, 5, 3} y termina como {7, 5, 3, 2}.
13	d.[70, 50, 30, 20]	El comparador devuelve b.compareTo(a), así que invierte el orden natural y deja los valores de mayor a menor. Arrays.toString(values) imprime el arreglo con corchetes y comas, por eso el resultado es [70, 50, 30, 20].
14	c.8,6,4,1	Collections.reverseOrder() devuelve un comparador que invierte el orden natural. Como los elementos son objetos Integer, podemos utilizarlo con Arrays.sort(). El arreglo original es {4, 8, 1, 6} y, después de ordenar de mayor a menor, queda {8, 6, 4, 1}. En el código, cada número se imprime en una línea diferente.
15	a.Nadia, Mariel, Elo, Alejandro	Collections.reverseOrder() invierte el orden alfabético natural de los String. Por eso los nombres quedan en orden descendente: primero Nadia, luego Mariel, después Elo y finalmente Alejandro.
16	a.import java.util.Comparator; public class AgeComparator implements Comparator<Person> {	La opción a compara las edades restando p1.getAge() - p2.getAge(), mientras que la opción c utiliza

	@Override
	public int compare(Person p1, Person p2) {
	return p1.getAge() - p2.getAge();
	}
	}
	c.import java.util.Comparator;
	public class AgeComparator implements Comparator<Person> {
	@Override
	public int compare(Person p1, Person p2) {
	return Integer.compare(p1.getAge(), p2.getAge());
	}
	}
		Integer.compare(p1.getAge(), p2.getAge()). Las dos ordenan de menor a mayor.

17	c.[7, 5, 1]	La lista empieza como [7, 1, 5, 1]. Como p está declarada como List sin especificar el tipo genérico y se pasa el entero 1, Java utiliza el método remove(int index), que elimina el elemento ubicado en el índice 1. Por eso elimina el primer 1 y la lista queda [7, 5, 1].
18	c.The output is true and MyStuff does NOT fulfill the Object.equals() contract.	El método devuelve true porque solamente comprueba si m.name no es null. Como m1.name vale "guitar", la condición se cumple, aunque m2.name sea "tv". El problema es que equals() debería comparar correctamente si los objetos son iguales, no solamente comprobar que uno de sus campos tenga un valor. Por eso, aunque devuelve true, está implementado incorrectamente.
19	c.4	Es 4 porque la clase MyKeys sobrescribe equals(), pero no sobrescribe hashCode(). Un HashMap utiliza el código hash para localizar las claves y después compara la igualdad cuando corresponde. Al no implementar ambos métodos, el mapa puede tratar objetos que deberían considerarse iguales como claves distintas. En este caso se agregan cuatro instancias de MyKeys.